

oscNext Level 4: Rebuilding the noise classifier

pass3 processing, variable validation, and the choice of BDT

Koray Gökçeler

September 13, 2026

L3 → L4 rebuilt for pass3 · noise BDT trained · the reference target reached

Reference throughout: *oscNext – Simulations and sample* (v00.07),
oscNext team, 13 October 2022.

THE OSCNEXT SELECTION CHAIN

The stages, as the reference document lists them

Stage	Objective, quoted
L2	"common collaboration filter level (we use the DeepCoreFilter)"
L3	"simple cuts to remove 'easy' noise, muon and unsimulated events and achieve good data-MC agreement before machine learning"
L4	"machine learning (BDT ensembles) to remove noise and muons"
L5	"removal of muons entering DeepCore via lightly-instrumented 'corridors' in the IceCube veto"
L6	"final level neutrino reconstruction (starting track+cascade hypothesis)"
L7	"final level muon and noise rejection cuts using final level reconstructions"

oscNext – Simulations and sample (v00.07), sec. 1.3

at L3	rate [mHz]
atmospheric muons	505
detector noise	36.6
neutrinos	5.25

oscNext – Simulations and sample (v00.07), Table 13

L4 inverts this ratio – with machine learning, and before any reconstruction, which the chain pays for only from L6 onward.

WHAT L4 DOES, AND WHAT HAD TO BE REBUILT

$$L4 = P_{\nu}^{\text{noise}} \geq 0.70 \wedge P_{\nu}^{\text{muon}} \geq 0.65$$

oscNext – Simulations and sample (v00.07), sec. 3.5

Why rebuild? The original segment is commented out and needs missing projects:

- ▶ `icecube.oscNext – I3Classifier`
- ▶ `tau_bdt – VICH`
- ▶ `analysis.event_selection – Dunkman variables`

Some of the inputs were **rewritten in pure Python**, from the reference document.

Where the BDT inputs come from

n	source	status
4	ready from L3	validated
5	IceTray projects	validated
5	our rewrite	unverified

oscNext – Simulations and sample (v00.07), Tables 11–12

All the risk is in the last row.

Compared line by line against the reference and the original pass2 segment. That comparison found bugs that **raise no error**.

NOISE BDT INPUTS, AS THE REFERENCE DEFINES THEM

Variable	Description
IC2018_LE_L3_Vars.NchCleaned	"See description in section 3.5.1" — there: "A number of hit DOMs in the cleaned pulse series."
L4_micro_count.STW_m3500p4000_DTW200	"Start with the cleaned pulse series. Look at pulses occurring within $[-3.5\mu\text{s}, +4\mu\text{s}]$ from the trigger time. Slide a time window of 200 ns that maximizes the number of triggered DOMs in it. Get the number of triggered DOMs in that sliding time window"
L4_iLineFit.speed	"Speed fitted by the improved LineFit algorithm"
L4_fill_ratio.fill_ratio_from_mean	"Measure of the geometrical spread of the hits about some vertex (details here)"
IC2018_LE_L3_Vars.FullTimeLengthRatio	"Measure the total duration of an event in both the cleaned and uncleaned pulse series (duration = max pulse time – min pulse time). Calculate the ratio between the two."

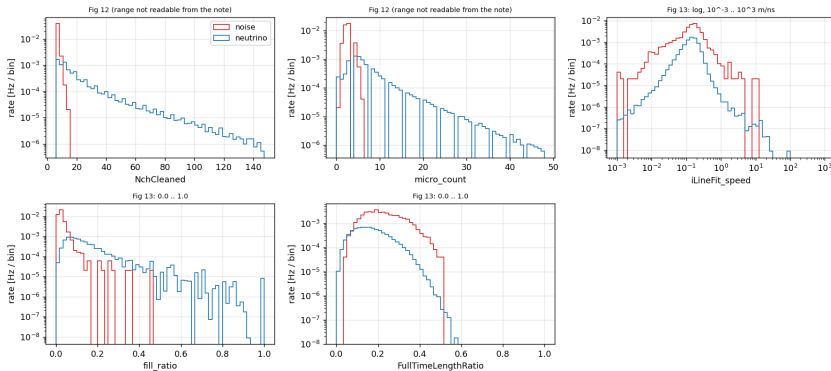
oscNext – Simulations and sample (v00.07), Table 11

Two of these are **not definitions**: fill_ratio's vertex is an unfilled cross-reference, and the ratio's direction is never stated. Both were recovered from the original pass2 code and from Figure 13.

NOISE BDT INPUTS, SIGNAL VS. NOISE

Our own pass3 processing

L4_noise inputs - rate vs variable (technical note Fig 12/13 style)

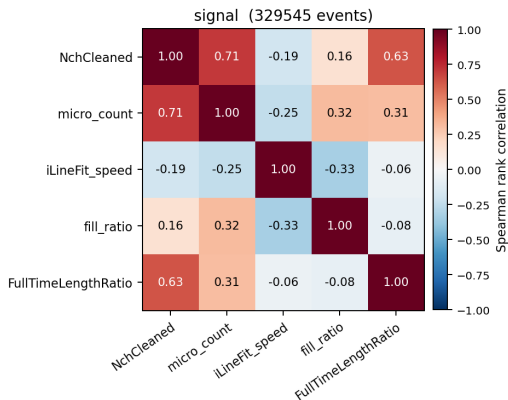


rate vs. variable for the five inputs

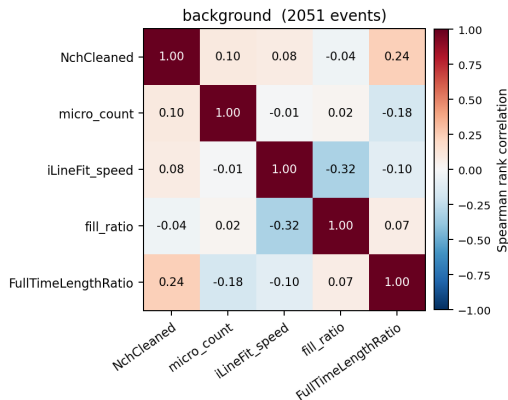
Measured, not assumed. `FullTimeLengthRatio` separates the *opposite* way to the expectation.

INPUT CORRELATIONS

Does any pair of the five inputs say the same thing?



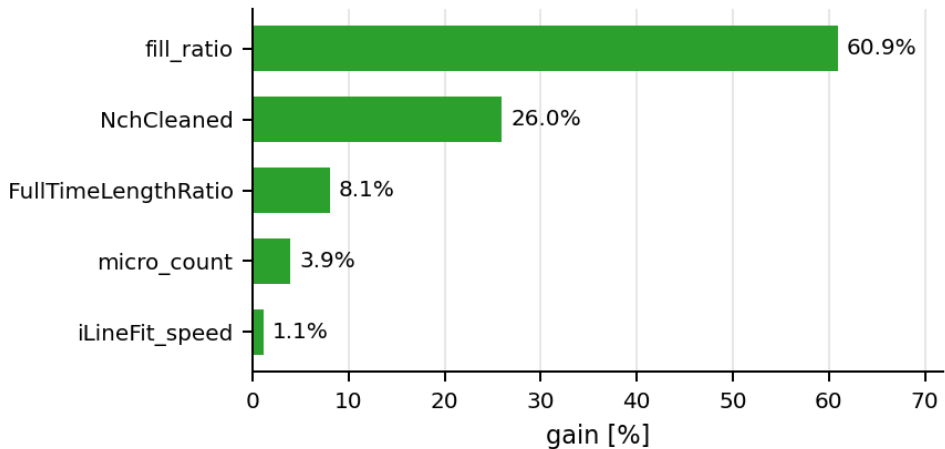
Spearman rank correlation, signal



Spearman rank correlation, background

FEATURE IMPORTANCE

Which input does the work?



HOW A BOOSTED DECISION TREE WORKS

Many shallow trees in sequence, each correcting the last.

The engines differ in *how* the next tree is chosen:

AdaBoost

(pybdt)

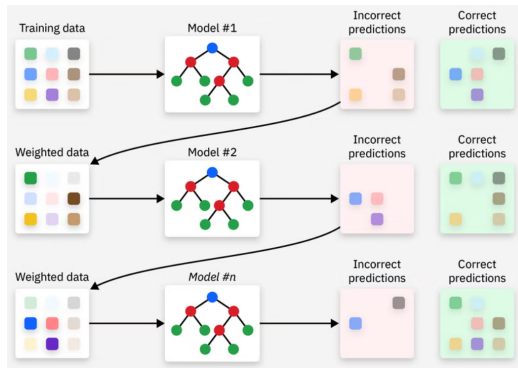
Gradient

boosting

(LightGBM)

re-weights the *events* it
gets wrong
fits the *gradient* of the
loss

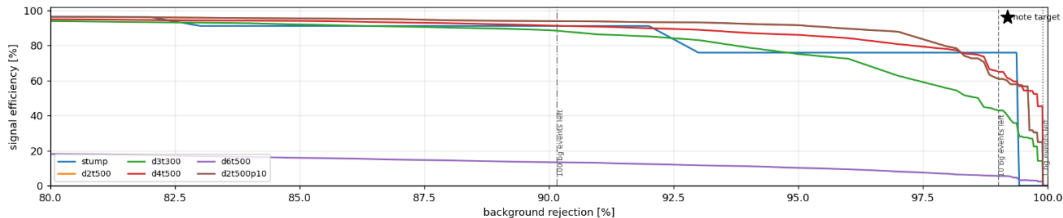
oscNext – Simulations and sample (v00.07), sec. 3.6.1



trees in sequence, leaf values summed

Output scale. LightGBM returns a probability, so the reference cut transfers to our model directly. A pybdt score does not.

TRAINING WITH PYBDT (ADABOOST)



six configurations, efficiency vs. rejection, test set

model	90 %	95 %	99 %
stump (d1)	91.3	76.0	76.0*
d2t500	94.0	91.7	62.0
d3t300	88.9	75.2	43.7
d4t500	91.5	86.1	65.8
d6t500	13.4	10.3	5.8

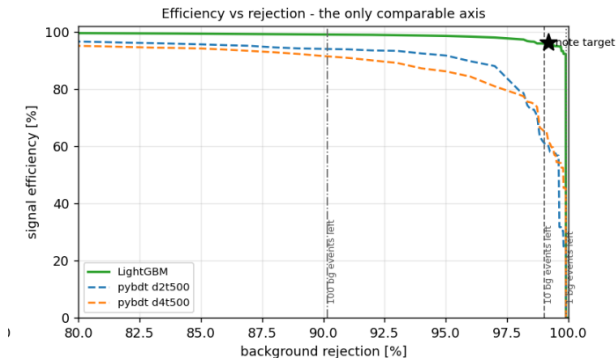
✗ the reference target is out of reach

✓ training is deterministic

✗ the KS p -value is useless here – we use the train/test efficiency *gap*

signal efficiency [%] at fixed noise rejection; * too few distinct scores at depth one

TRAINING WITH LIGHTGBM



same events, same variables, same split

rejection	AdaBoost	LightGBM
90 %	94.0	99.0
95 %	91.7	98.5
99 %	65.8	95.9

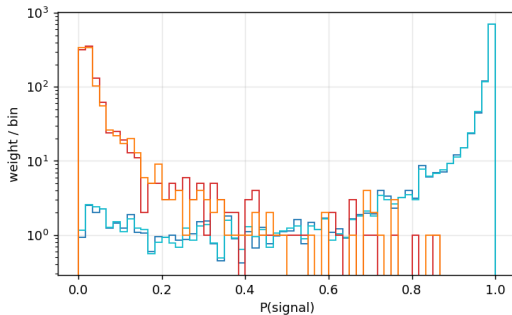
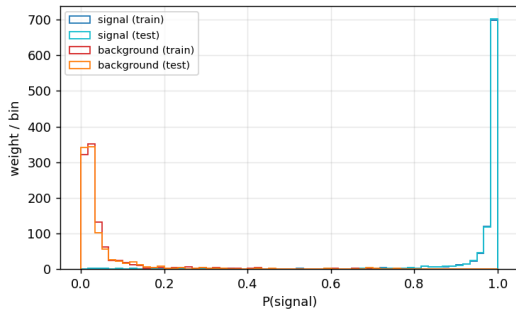
signal efficiency [%], test set, event counts

- ✓ the reference target is reached
- ✓ no overtraining – train and test coincide
- ✓ reached while *handicapped* by the reference leaf-size floor

LIGHTGBM SCORE DISTRIBUTION

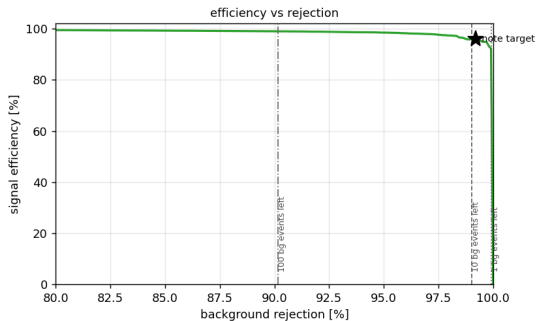
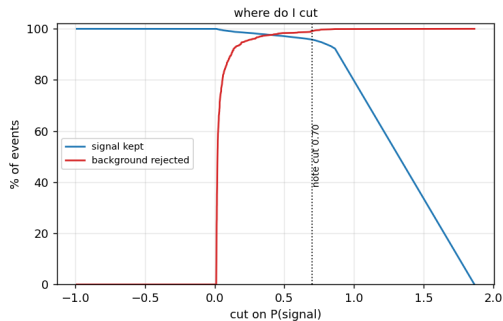
The overtraining check

noise - score distribution (weight: weight)



THE LIMIT: NOISE MC STATISTICS

noise - cut curves, efficiency counted in events



STATUS AND NEXT STEPS

Done

- ▶ L3 → L4 rebuilt for pass3, robust against corrupt files
- ▶ all BDT inputs located
- ▶ validated against the reference and the original segment
- ▶ noise classifier trained, target reached

Next

1. **Cross-check against pass2 L4 files** – validates the rewritten variables; tests each model on the other pass
2. **Re-tune the fill_ratio radius** – carries most of the gain (see the importance plot), never re-optimised for oscNext
3. **Muon classifier** – CORSIKA background
4. **More vuvuzela MC** – to measure the far tail
5. Reprocess the remaining samples